Digital Chemical Engineering 13 (2024) 100182

Contents lists available at ScienceDirect

Digital Chemical Engineering

journal homepage: www.elsevier.com/locate/dche

Original article

BibMon: An open source Python package for process monitoring, soft sensing, and fault diagnosis

Afrânio Melo [a,b,*], Tiago S.M. Lemos [a,c], Rafael M. Soares [a], Deris Spina [a], Nayher Clavijo [a], Luiz Felipe de O. Campos [a], Maurício Melo Câmara [a,d], Thiago Feital [a,d], Thiago K. Anzai [b], Pedro H. Thompson [b], Fábio C. Diehl [b], José Carlos Pinto [a]

[a] Programa de Engenharia Química/COPPE, Universidade Federal do Rio de Janeiro, CEP 21941-972, Rio de Janeiro, RJ, Brazil
[b] CENPES, Centro de Pesquisas Leopoldo Américo Miguez de Melo, Petrobras — Petróleo Brasileiro SA, CEP 21941-915, Rio de Janeiro, RJ, Brazil
[c] Gestão de Parcerias e Processos de Exploração e Produção (GPP-E&P), Petrobras — Petróleo Brasileiro SA, CEP 20231-030, Rio de Janeiro, RJ, Brazil
[d] OPTIMATECH Ltda., Rio de Janeiro, RJ, CEP 22410-905, Brazil

ARTICLE INFO

Keywords:
Process monitoring
Fault detection and diagnosis
Soft sensor
Machine learning
Open source

ABSTRACT

This paper introduces BibMon, a Python package that provides predictive models for data-driven fault detection and diagnosis, soft sensing, and process condition monitoring. Key features include regression and reconstruction models, preprocessing pipelines, alarms, and visualization through control charts and diagnostic maps. BibMon also includes real and simulated datasets for benchmarking, comparative performance analysis of different models, and hyperparameter tuning. The package is designed to be highly extensible, allowing for easy integration of new models and methodologies through its object-oriented implementation. Currently, BibMon is in production at Petrobras, a major player in the energy industry, monitoring numerous industrial assets and enabling real-time detection and diagnosis of equipment and process faults. The software is open source and available at: https://github.com/petrobras/bibmon.

1. Introduction

In the context of Industry 4.0, digital transformation creates new opportunities for applying artificial intelligence in process industries. Specifically, process monitoring methodologies are of growing importance, as they ensure that processes operate under normal conditions. Key applications of process monitoring include fault detection and diagnosis, where anomalies and deviations from normal operation are captured and identified (Bi et al., 2022), as well as soft sensing, which employs algorithms to estimate variable values based on measurements of other variables (Perera et al., 2023).

The extensive literature on process monitoring covers a range of topics, including model development, process data exploration, real-world industrial implementations, and the availability of software and benchmarks. Recently, Melo et al. (2024) provided a comprehensive survey encompassing these topics. In particular, software development has been identified as a crucial component of the process monitoring activity. However, research in this specific topic remains notably scarce, especially when compared to the abundant literature on new mathematical models aimed at capturing specific characteristics of data, such as time dependence, nonlinearity and non-Gaussianity. While these proposed techniques hold increasing potential for real-world applications, they are often underutilized in industrial settings. In this context, Thibault et al. (2023) assert that most process monitoring applications in real industrial plants rely on commercial software solutions, which often lack connection to cutting-edge academic advancements disseminated via research articles.

Open source software development shows significant promise in bridging the gap between theoretical methodologies and practical implementations (Melo et al., 2024). Universities and research centers, with this perspective, have the capacity to create open source software solutions accessible to both academic and industry professionals. By doing so, these institutions can facilitate the adoption of advanced techniques in industrial applications, effectively translating academic research into real-world contexts. This approach finds widespread adoption in the fields of data science and machine learning, as exemplified by cutting-edge software libraries and suites such as scikit-learn (Pedregosa et al., 2011) and Hugging Face Hub (Wolf et al., 2020).

In the field of process systems engineering, open source tools for process monitoring have typically been developed as extensions of the

* Corresponding author at: CENPES, Centro de Pesquisas Leopoldo Américo Miguez de Melo, Petrobras — Petróleo Brasileiro SA, CEP 21941-915, Rio de Janeiro, RJ, Brazil.
E-mail address: afranio.melo@petrobras.com.br (A. Melo).

https://doi.org/10.1016/j.dche.2024.100182
Received 29 June 2024; Received in revised form 25 August 2024; Accepted 2 September 2024
Available online 2 September 2024
2772-5081/© 2024 The Authors. Published by Elsevier Ltd on behalf of Institution of Chemical Engineers (IChemE). This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Matlab platform. Notable examples include: MEDA, a toolbox designed for exploratory data analysis using multivariate statistical tools (Camacho et al., 2015); DB-KIT, which utilizes regression and reconstruction models to handle key performance indicators (Jiang et al., 2016; Jiang and Yin, 2019); MVBatch, a graphical user interface for batch process understanding, monitoring, and troubleshooting (González-Martínez et al., 2018); Pre-Screen, a toolbox that pre-processes process data (Yi et al., 2019); and LAVADE, a toolbox for understanding multivariate statistical latent variable models (Schaeffer and Braatz, 2022). Despite these tools being open source, Matlab's closed source code, proprietary nature, and licensing requirements hinder their compatibility with the open innovation paradigm. Consequently, the literature lacks tools developed in a fully open environment that can be readily serve as platforms for bridging academic advancements and industrial implementations.

This paper presents BibMon, an open source Python library that provides predictive models for fault detection and diagnosis, soft sensing, and process condition monitoring. Currently, BibMon offers the following features: regression and reconstruction models, preprocessing pipelines, alarms, visualization through control charts and diagnostic maps, real and simulated datasets for benchmarking, comparative performance analysis of different models, and hyperparameter tuning. Additionally, the library is highly extensible, allowing for easy integration of new models and methodologies due to its object-oriented implementation. The Python language was a natural choice for development due to its widespread adoption in the data science domain, making it an ideal platform for collaborative efforts within the open innovation paradigm. The software has been registered with the Brazilian intellectual property office (Melo et al., 2023) and its repository can be accessed using the following link: https://github.com/petrobras/bibmon.

BibMon is currently being used to monitor several industrial assets at Petrobras, enabling real-time detection and diagnosis of equipment and process faults (Anzai et al., 2023). The library is part of the Open Lab Module of Petrobras' Connections for Innovation program, which aims to integrate Petrobras with the open innovation community. In particular, the Open Lab Module promotes cooperation to address problems in the oil, gas, and energy industry through open source software projects. Other components of Open Lab, such as the 3W Project (Vargas et al., 2019), can be accessed in https://github.com/petrobras.

This paper is structured as follows. Section 2 details the structure of the package and the ways in which it can be extended. Section 3 presents illustrative results obtained using the library with simulated and real data. Section 4 concludes the article.

2. Methodology

2.1. Deviation-based process monitoring

Several methodologies have been developed to enable operators to track process conditions. Two commonly used methodologies are statistical process control and deviation-based process monitoring. Statistical process control utilizes control charts to monitor statistical parameters such as mean, standard deviation, and range of individual variables (Montgomery, 2024). This approach helps in detecting special cause variations and potential process anomalies. Another commonly used approach is deviation-based process monitoring, which focuses on monitoring deviations from expected values or patterns (Melo et al., 2024). Algorithms can be designed to compare actual sensor measurements with model representations to identify process anomalies. BibMon is based on deviation-based methodologies.

Deviation-based methodologies can be categorized into two main groups: reconstruction and regression techniques, depending on how they construct models to represent normal process behavior. Both techniques capture the key patterns of datasets during the training phase in order to build models. Once model training is complete, new test data can be compared to the identified model structure, and deviations are used to detect anomalies in the process.

Reconstruction techniques aim to represent normal process behavior by utilizing a mathematical structure that incorporates all relevant variables simultaneously. The objective is typically to create a compressed representation of the data that still captures the essential information, often involving dimensionality reduction methods. During the test phase, the data is compared to the identified structure to detect any deviations. In practice, a reconstruction model denoted as F accepts a data matrix $\mathbf{X}$ and returns a reconstructed version $\hat{\mathbf{X}}$ with the same shape, according to the relationship:

$$\hat{\mathbf{X}} = F(\mathbf{X}) \tag{1}$$

The deviation can then be defined by calculating the distance between $\mathbf{X}$ and $\hat{\mathbf{X}}$. A commonly used metric is the square prediction error (SPE), also known as the Q index, which is defined as:

$$\mathrm{SPE}_i = \sum_j (X_{ij} - \hat{X}_{ij})^2 \tag{2}$$

where i refers to observations and j refers to variables. The SPE can be monitored to check if the reconstruction error is below an acceptable threshold. Examples of reconstruction models include principal component analysis (PCA), which reduces dimensionality while preserving most of the data variance (Melo et al., 2024), and autoencoders, which employ neural networks to encode input data into a compressed representation before reconstructing the output to mirror the input (Spina et al., 2024).

Regression techniques, on the other hand, establish relationships between input variables (also known as predictors) and output variables (also known as predictions). By dividing the dataset into a predictor matrix $\mathbf{X}$ and a predicted matrix $\mathbf{Y}$, a regression model denoted as f can be expressed as:

$$\hat{\mathbf{Y}} = f(\mathbf{X}) \tag{3}$$

In this case, the SPE would only consider the predictions:

$$\mathrm{SPE}_i = \sum_j (Y_{ij} - \hat{Y}_{ij})^2 \tag{4}$$

It should be noted that the measured output matrix $\mathbf{Y}$ is only utilized during the training phase. In the test phase, predictions for $\hat{\mathbf{Y}}$ rely exclusively on information from $\mathbf{X}$. Therefore, this methodology is associated with the concept of soft sensing, as the model effectively acts as a virtual sensor for the output variables.

In BibMon, the similarities between regression and reconstruction methodologies are utilized to develop a generic template that supports the implementation of both types of techniques. This was achieved using object-oriented programming, a technique detailed in the next subsection.

2.2. Object-oriented programming

BibMon was designed for easy extensibility and maintenance, making the code accessible even to users with little software development experience. This user-friendly design is primarily due to its foundation in object-oriented programming (OOP). In OOP, data and the functions that manipulate this data are grouped into computational entities known as objects. This encapsulation simplifies data abstraction, enhancing complexity management beyond mere process abstraction (Sebesta, 2018). Unlike procedural programming, OOP centers on data: each object contains its own data (attributes) and specific functions (methods) designed to operate on that data. OOP is widely adopted in the software industry (Lian et al., 2022; Pazel, 2022). It has been applied in process systems engineering, for example, to compute thermodynamic properties using equations of state (Shi and

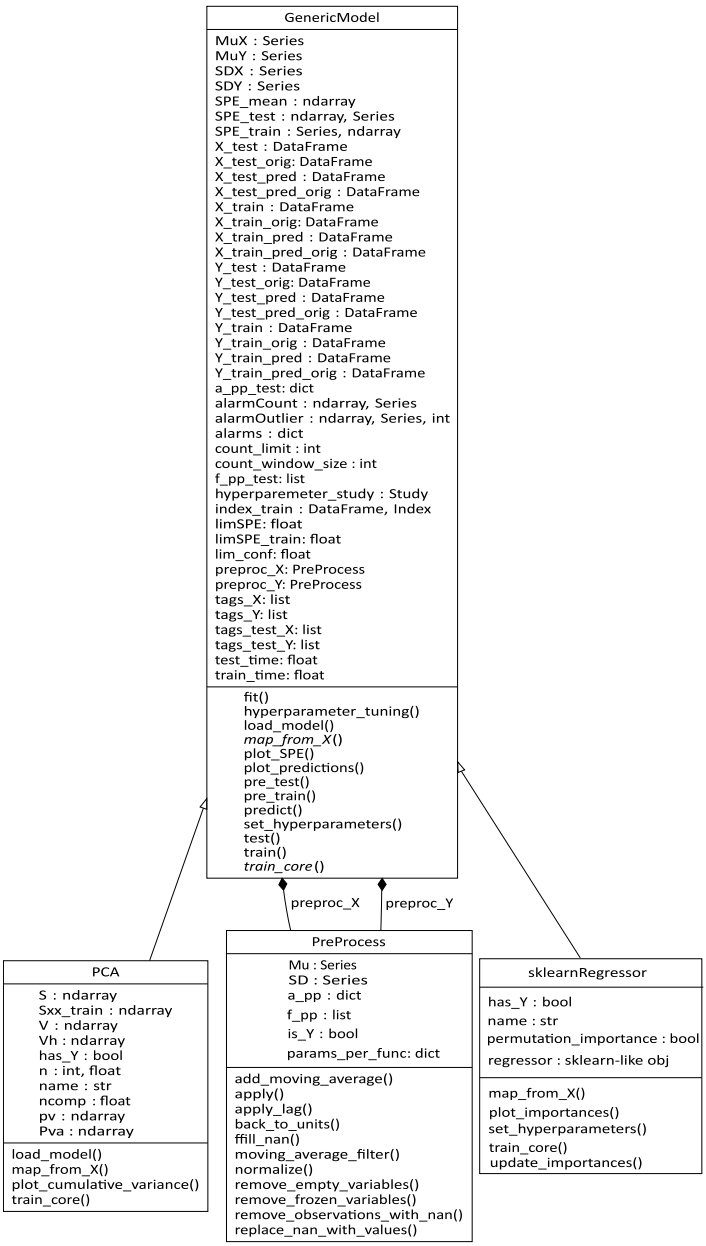

Fig. 1. BibMon UML diagram.

Liang, 2011) and to model different configurations of membrane gas separation units (Fontoura et al., 2022).

Classes and their relationships can be depicted in a representation known as UML diagram (Rumbaugh et al., 2005). Fig. 1 shows the UML diagram for BibMon. This figure was automatically generated from the source code using the pyreverse tool from the pylint Python package and adapted with minor adjustments. The components depicted in this diagram will be explained in the following subsections. For a complete reference of the terms shown in the diagram, please refer to the official documentation of the package at https://bibmon.readthedocs.io.

2.3. Models

Reconstruction and regression models to represent process behavior are implemented using a class called GenericModel. This class encompasses all the common data and functionalities shared by models, including training and test data, alarms, squared prediction error (SPE), statistical summaries of central tendency (MuX and MuY) and dispersion (SDX and SDY), hyperparameter optimization, among other elements. It is important to note that GenericModel is an abstract class, meaning that it cannot be instantiated directly. Instead, specific models like PCA and sklearnRegressor are derived from GenericModel using inheritance, a concept in object-oriented programming. In this relationship, the derived classes inherit all the functionalities from the parent class and also implement their own specific functionalities. For example, the PCA class implements a method called plot_cumulative_variance, while the sklearnRegressor class has an attribute called regressor, as depicted in Fig. 1. To maintain visual clarity and facilitate comprehension, other methods available in BibMon, such as echo state networks (Lemos et al., 2021), autoencoders (Spina et al., 2024), and similarity-based methods (Marins et al., 2018), have not been included in Fig. 1.

The definition of GenericModel as an abstract class lies in the presence of abstract methods, which are methods without an implementation. In Fig. 1, abstract methods are denoted in italic font. Two

such methods, `map_from_X()` and `train_core()`, can be seen. The `map_from_X()` method specifies the function responsible for taking an input data matrix $\mathbf{X}$ and mapping it to the output of the model, which can be either the reconstructed matrix $\hat{\mathbf{X}}$ or the regressed matrix $\hat{\mathbf{Y}}$. On the other hand, the `train_core()` method outlines the core of the training algorithm, encompassing all the necessary steps between the preparation step (handled by the `pre_train()` method) and the calculation of the prediction or reconstruction during training, achieved through the `map_from_X()` method. When deriving new models from the `GenericModel` template, developers are required to implement these two methods. Other functionalities can be implemented as needed.

2.4. Data preprocessing and treatment

Data preprocessing is handled by the `PreProcess` class, which contains various preprocessing methods such as normalization, removal of NaN observations, and more. These methods are responsible for preparing the raw data to be fed into the training models. The `apply` method in the `PreProcess` class is responsible for applying the specified preprocessing methods sequentially, as defined during the initialization of a `PreProcess` object.

The `PreProcess` class is related to the `GenericModel` class through a composition relationship. In this relationship, the `GenericModel` class has attributes that are instances of the `PreProcess` class. These instances are utilized internally by the `GenericModel` to execute the preprocessing functions during the preparation steps before training or testing. Users specify which preprocessing functions must be executed when invoking training or test functions, as detailed in the examples of Section 3.

To introduce new preprocessing techniques, developers should implement them as methods of the `PreProcess` class, following the template below:

```
def new_method(self, df, train_or_test='train'):
    if self.is_Y:
        # code generating a processed df if the data is Y
        if train_or_test == 'train':
            # code generating a processed df if the data is for training
    if ...
    return processed_df
```

Code snippet 1: Template for new preprocessing methods.

It is worth noting that when implementing new functions, developers have the flexibility to consider different scenarios based on the nature of the data (predictors or predictions) and the analysis stage (training or testing), depending on their specific requirements. This is achieved through the use of the flags `self.is_Y` and `train_or_test`, respectively, as shown in Code snippet 1.

2.5. Datasets

To facilitate the rapid testing of new functionalities in `BibMon`, datasets have been included in the software. Currently, users have access to two datasets: the well-known Tennessee Eastman Process dataset (Downs and Vogel, 1993; Chiang et al., 2001; Melo et al., 2022) and an anonymized dataset with actual measurements from a real process plant. The software also provides specific functions to load these datasets. The tutorials available in the documentation are built around these datasets, allowing users to gain hands-on experience and explore the capabilities of the software. Additionally, the contributing guide outlines the process for adding new datasets to `BibMon`.

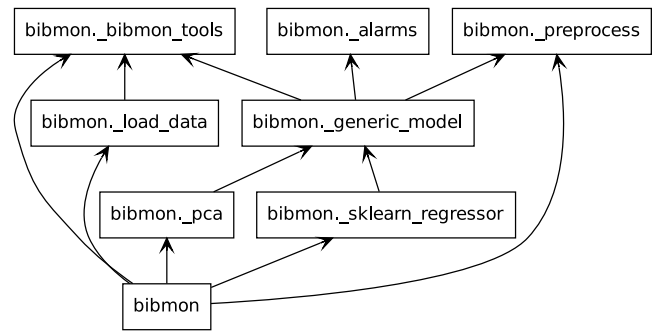

Fig. 2. Structure of the `BibMon` package.

2.6. Alarms

In `BibMon`, developers have the flexibility to implement various alarm logics for monitoring the SPE index. The library currently offers support for outlier detection, allowing users to set a specific limit for detecting outliers. This can be done either by considering individual observations or by counting the number of outliers within a specified window.

2.7. Other functionalities

Several additional functionalities have been implemented in a specific file named `bibmon_tools.py`. These functionalities include functions for conducting automated offline analysis, plotting control charts, and creating comparative tables to facilitate performance analysis of different models. Developers are encouraged to utilize this file to incorporate supplementary functionalities into the software.

2.8. Package structure

Fig. 2 shows the structure of the `BibMon` package. The main module, `bibmon`, includes submodules for different tasks: data modeling (`_pca` and `_sklearn_regressor`), data loading (`_load_data`), data preprocessing (`_preprocess`), and utility functions (`_bibmon_tools`). The `_generic_model` submodule is at the center of the figure, highlighting its key role in the code structure. It serves as the base for specific models like `_pca` and `_sklearn_regressor`, and utilizes other modules as well. Fig. 2 was automatically generated from the source code using the `pyreverse` tool from the pylint Python package.

3. Results and discussion

Examples of applications of `BibMon` will be provided using the two datasets currently included in the package: the TEP dataset and a real process plant dataset. These examples are also explained in detail in the tutorials available as Jupyter notebooks in the package repository. The examples will be shown as code snippets, along with the figures generated by the respective codes.

3.1. TEP application

Code snippet 2 shows the commands used to train a PCA model with the normal TEP dataset and plot the cumulative variance of the data. In line 4, the function `load_tennessee_eastman` is used to load the training and test data into `pandas` DataFrames (McKinney, 2010). Next, a model is instantiated (line 6) and trained (line 7). Finally, the cumulative variance is plotted (Fig. 3), with a vertical line indicating the number of components retained in the dimensionality

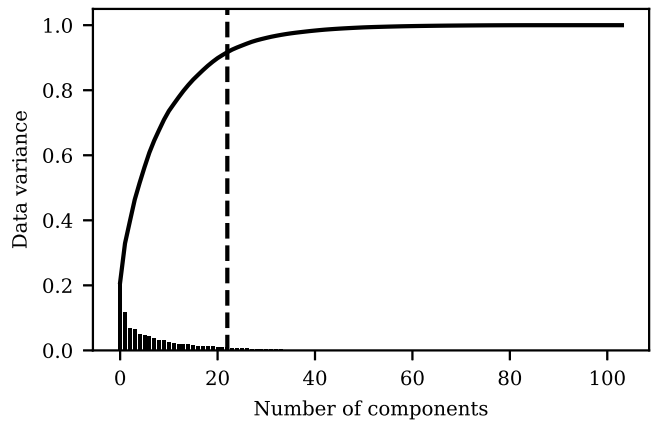

Fig. 3. Variances computed by the PCA model using the TEP training data. Bars represent the data variance captured by each component, while the continuous line shows the cumulative variance. The vertical dashed line indicates the number of principal components retained by the model.

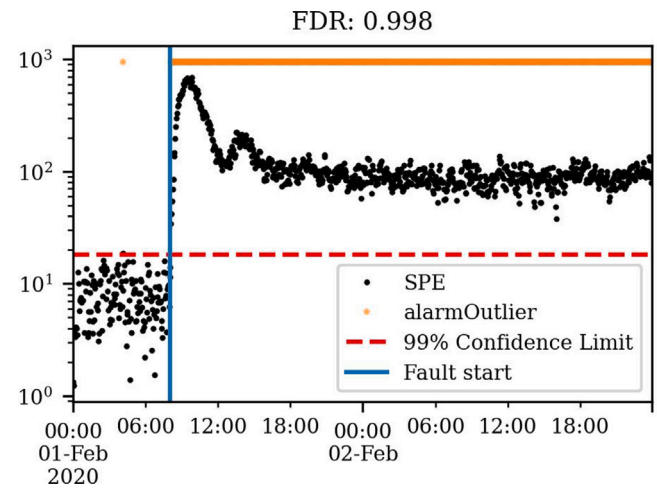

Fig. 4. PCA control chart for event 1 of the TEP dataset.

reduction step. It is worth noting that the API is inspired by `scikit-learn` (Pedregosa et al., 2011), which makes it easy for those familiar with the Python machine learning ecosystem to use BibMon fluently. This follows the common practice in the industry.

```
import bibmon
import matplotlib.pyplot as plt

df_train, df_test = bibmon.load_tennessee_eastman(
    train_id = 0, test_id = 1)

model = bibmon.PCA()
model.fit(df_train, redefine_limit = True, frac_val =
    0.1)

model.plot_cumulative_variance()
plt.axvline(model.n, ls='--')
```

Code snippet 2: PCA model training.

Code snippet 3 demonstrates how to test the trained PCA model. It is important note that, although the function used is technically a reconstruction rather than a prediction, the term "predict" is maintained in the function's nomenclature to ensure consistency with the style of `scikit-learn`. The first line is dedicated to the test and the next ones are related to plotting results. In the last line, the fault detection rate (FDR) value is computed using the `alarmOutlier` attribute stored in the model object. Fig. 4 shows the SPE control chart generated by Code snippet 3.

```
model.predict(df_test)

model.plot_SPE(train_or_test = 'test')
plt.axvline(fault_start, label='Fault start')
plt.title(f'FDR: {model.alarmOutlier[fault_start:].
    mean()}')
```

Code snippet 3: PCA model test.

Code snippet 4 demonstrates the training of four PCA models using test data related to fault 11 of TEP. These models vary based on the data pretreatment steps employed. All models include data normalization, as required by PCA. However, they differ in the combinations of the `apply_lag` and `moving_average_filter` methods used. The final model incorporates personalized values for the arguments of the pretreatment functions. The corresponding results are visualized in Fig. 5, illustrating how different combination of preprocessing methodologies can improve fault detection metrics.

```
import bibmon
import matplotlib.pyplot as plt

fig, ax = plt.subplots(2, 2)

df_train, df_test = bibmon.load_tennessee_eastman(
    train_id=0, test_id=11)

models = [bibmon.PCA() for _ in range(4)]

preproc1 = ['apply_lag', 'normalize']
preproc2 = ['moving_average_filter', 'normalize']
preproc3 = ['apply_lag', 'moving_average_filter', '
    normalize']
args_preproc3 = {'moving_average_filter__WS': 5, '
    apply_lag__lag': 5}

preprocessing_steps = [
    {},
    {'f_pp': preproc1, 'f_pp_test':preproc1},
    {'f_pp': preproc2, 'f_pp_test': preproc2},
    {'f_pp': preproc3, 'f_pp_test': preproc3, 'a_pp':
      args_preproc3, 'a_pp_test': args_preproc3}
]

val = {'redefine_limit': True, 'frac_val': 0.1}

for model, preproc in zip(models, preprocessing_steps
    ):
    model.fit(df_train, **preproc, **val)

for i, (model, axis) in enumerate(zip(models, ax.
    flatten())):
    model.predict(df_test)
    model.plot_SPE(train_or_test='test', ax=axis)
    axis.axvline(fault_start, label='Fault start')
    axis.set_title(f'Model {i + 1}\nFDR: {model.
    alarmOutlier[fault_start:].mean():.3f}')
```

Code snippet 4: Training of a PCA model with different preprocessing techniques.

3.2. Real process plant application

Code snippet 5 presents the necessary commands to execute the complete process monitoring pipeline for multiple models, with the

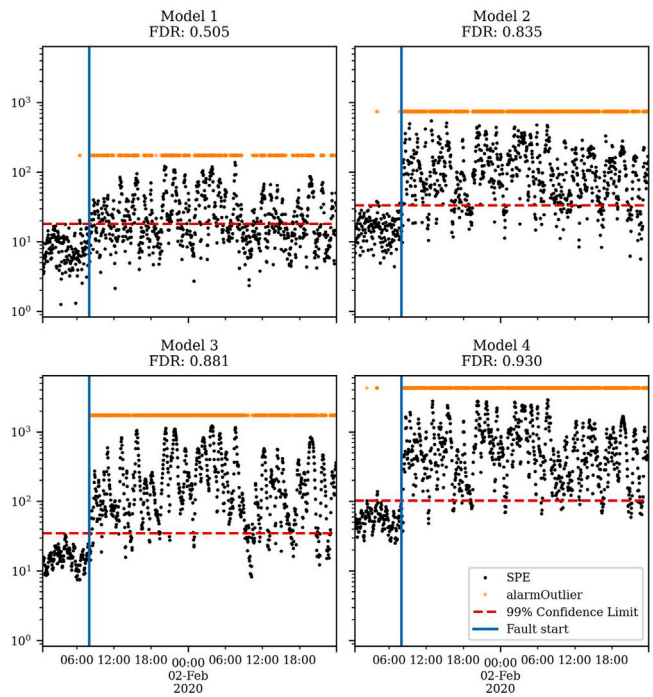

Fig. 5. PCA control charts for event 11 of the TEP dataset.

goal of comparing their results. In line 10, the dataset with a fault from an actual process plant is loaded. Lines 12–14 handle the preprocessing of raw non-numeric data and split the dataset into training, validation, and test periods. Lines 16–18 instantiate the imported `scikit-learn` regression models as `BibMon` objects. Finally, in line 20, the high-level function `comparative_table` is used to execute all the steps in the monitoring pipeline and generate Figs. 6–7 and Tables 1–3.

```
1  import bibmon
2  import pandas as pd
3  from sklearn.linear_model import LinearRegression
4  from sklearn.linear_model import LassoCV
5  from sklearn.svm import SVR
6  from sklearn.neighbors import KNeighborsRegressor
7  from sklearn.ensemble import RandomForestRegressor
8  from sklearn.neural_network import MLPRegressor
9
10 df_raw = bibmon.load_real_data()
11
12 df_num = df_raw.apply(pd.to_numeric, errors = 'coerce
       ')
13
14 (X_train, X_validation, X_test, Y_train,
       Y_validation, Y_test) = bibmon.
       train_val_test_split(df_num, start_train = '
       2017-12-24T12:00', end_train = '2018-01-01T00:00'
       , end_validation = '2018-01-02T00:00', end_test
       = '2018-01-04T00:00', tags_Y = 'tag100')
15
16 sklearn_regs = [LinearRegression(), LassoCV(), SVR(),
       RandomForestRegressor(n_jobs=-1,random_state=1),
       MLPRegressor()]
17
18 models = [bibmon.sklearnRegressor(reg) for reg in
       sklearn_regs]
19
```

Table 1
Performance metrics for different models.

Models	Metrics	Train	Validation	Test
LassoCV	mean_absolute_error	0.051	0.058	0.190
	r2_score	0.993	0.940	−0.865
LinearRegression	mean_absolute_error	0.047	0.043	0.561
	r2_score	0.994	0.959	−1.757
MLPRegressor	mean_absolute_error	0.038	0.313	0.754
	r2_score	0.997	−0.870	−2.352
RandomForestRegressor	mean_absolute_error	0.012	0.025	0.144
	r2_score	1.000	0.986	−0.335
SVR	mean_absolute_error	0.060	0.143	0.533
	r2_score	0.990	0.567	−0.515

Table 2
FDR and FAR for different models and alarm types.

Alarms	Models	FDR	FAR
alarmCount WS = 3, limCount = 2	LassoCV	0.778	0.019
	LinearRegression	0.778	0.720
	MLPRegressor	0.278	0.357
	RandomForestRegressor	0.750	0.007
	SVR	0.472	0.226
alarmOutlier	LassoCV	0.833	0.037
	LinearRegression	0.833	0.741
	MLPRegressor	0.444	0.389
	RandomForestRegressor	0.806	0.039
	SVR	0.528	0.269

```
20 tab_pred, tab_detec, tab_times = bibmon.
       comparative_table(models, X_train, X_validation,
       X_test, Y_train, Y_validation, Y_test, metrics =
        mtr, count_window_size = 3, count_limit = 2,
       fault_start = '2018-01-02 06:00:00', fault_end =
       '2018-01-02 09:00:00')
```

Code snippet 5: Executing the full process monitoring pipeline with the goal of comparing model results.

The plots in Fig. 6 depict the actual and predicted values of the regressed variable. The SPE control charts resulting from the differences between the actual and predicted values are shown in Fig. 7. In both Figs. 6 and 7, the blue dashed vertical lines indicate the start and end moments of the faulty event. The numerical results associated with these charts are presented in Tables 1 and 2. The fitting metrics in Table 1 indicate that the Random Forest model (Breiman, 2001) performs the best. This is further supported by Table 2, which demonstrates that the Random Forest model achieves a high fault detection rate with a low false alarm rate. However, it is worth noting that this superior performance comes at the expense of computational efficiency, as evidenced in Table 3. The model is one of the slowest among the evaluated models. It is important to emphasize that all these results, involving multiple computational steps that compose typical process monitoring pipelines, can be generated with just a few lines of code using `BibMon`.

In addition to its offline data analysis capabilities, `BibMon` is also designed for online use. It is the most widely used backend library in the SmartMonitor monitoring platform, an in-house application developed by Petrobras (Anzai et al., 2023). This platform allows engineers to easily create real-time monitoring tasks without requiring coding expertise, with setup taking only a few minutes. Currently, `BibMon` is being utilized for 136 monitoring tasks, some of which include:

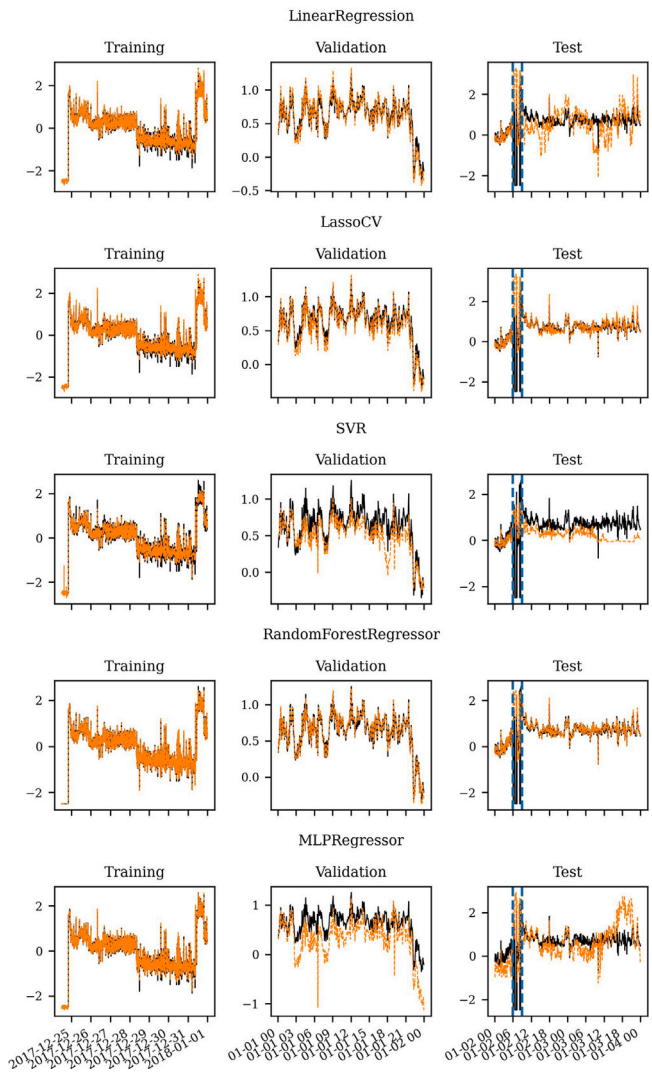

Fig. 6. Representation of the predicted variable across training, validation, and test sets (black continuous line for measurements, dashed orange lines for predictions). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

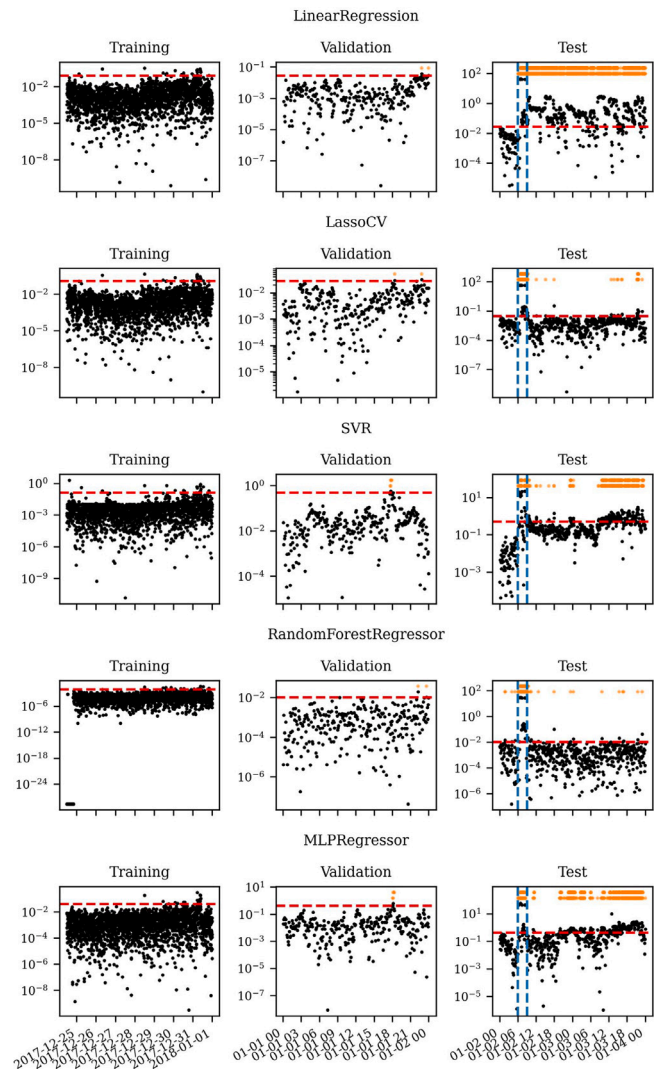

Fig. 7. SPE control charts for training, validation, and test for multiple models. Two types of alarms are represented by orange points: the superior points indicate the `alarmCount`, which considers time windows, while the inferior points represent the simpler `alarmOutlier`. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 3
Training and test execution times for different models (measured in seconds per million processed observations).

Models	Train	Test
LinearRegression	15.74	0.59
LassoCV	91.67	0.54
SVR	131.06	68.60
RandomForestRegressor	455.02	63.65
MLPRegressor	966.12	1.48

- virtual sensor for estimating the fiscal flow rate of produced oil;
- virtual sensor for measuring oil and grease content in produced water;
- virtual sensor for monitoring gas emissions in turbogenerators within natural gas processing units;
- monitoring of gas flow meters for flare operations;
- monitoring of humidity meters in natural gas dehydration systems;
- monitoring of advanced control parameters;
- monitoring of compression systems.

By combining SmartMonitor with `BibMon`, Petrobras has successfully detected and diagnosed faults in the aforementioned applications, resulting in substantial cost savings and reduced production losses.

4. Conclusion

This paper presented `BibMon`, a Python package designed for data-driven fault detection and diagnosis, soft sensing, and process monitoring. With a wide range of features and the flexibility in incorporating new models and methodologies, `BibMon` is a valuable tool for monitoring industrial assets. Additionally, as an open-source package, `BibMon` promotes collaboration and knowledge sharing within the industry and facilitates the integration of cutting-edge developments from academia into industrial settings. Its successful implementation at Petrobras demonstrates its efficacy in real-time fault detection and diagnosis. Overall, `BibMon` aims to be a reference software, assimilating

and making accessible various methodologies for data-driven process monitoring and analysis.

As a software continuously developed within the open innovation community, BibMon can be enhanced to address certain shortcomings identified in its practical applications:

- Classification-based applications: Some applications would benefit from classification models rather than regression or reconstruction models. To address this need, a classification module has been added to the package backlog.
- Advanced reconstruction methods: Advanced versions of techniques such as PCA and PLS should be explored and implemented to enhance fault diagnosis accuracy, particularly when considering specific data characteristics like nonlinearity, dynamics, and non-Gaussianity (Melo et al., 2024).
- Interpretability: To enhance interaction and adoption by operators, models need to be interpretable. The implementation of interpretability techniques such as LIME (local interpretable model-agnostic explanations) (Ribeiro et al., 2016) and SHAP (Shapley additive explanations) (Lundberg and Lee, 2017) is also on the roadmap.
- Retraining frequency: The optimal frequency for retraining the models used by BibMon needs to be determined to ensure accuracy. Research is required to establish the ideal retraining intervals for each model.

Improving these areas will further enhance the effectiveness of BibMon as a reference for process monitoring in the open-source ecosystem.

Funding

This research was funded in part by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001.

CRediT authorship contribution statement

Afrânio Melo: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tiago S.M. Lemos:** Writing – review & editing, Validation, Software, Methodology, Formal analysis, Data curation. **Rafael M. Soares:** Validation, Software, Methodology, Conceptualization. **Deris Spina:** Validation, Software, Formal analysis, Conceptualization. **Nayher Clavijo:** Methodology, Formal analysis, Conceptualization. **Luiz Felipe de O. Campos:** Methodology, Formal analysis, Conceptualization. **Maurício Melo Câmara:** Writing – review & editing, Software, Methodology, Conceptualization. **Thiago Feital:** Writing – review & editing, Software, Methodology, Formal analysis, Conceptualization. **Thiago K. Anzai:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Funding acquisition. **Pedro H. Thompson:** Software, Resources, Methodology. **Fábio C. Diehl:** Supervision, Resources, Project administration. **José Carlos Pinto:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Anzai, T.K., Furtado, P.H.T., de Brito, G.M., Santos, J.S., Moreira, P.C.M., Diehl, F.C., Ferreira, L.E.L., Grava, W.M., 2023. Catching Failures in 10 Minutes: An Approach to No Code, Fast Track, AI-Based Real Time Process Monitoring. OnePetro, http://dx.doi.org/10.4043/32898-MS.

Bi, X., Qin, R., Wu, D., Zheng, S., Zhao, J., 2022. One step forward for smart chemical process fault detection and diagnosis. Comput. Chem. Eng. 164, 107884. http://dx.doi.org/10.1016/j.compchemeng.2022.107884, URL: https://linkinghub.elsevier.com/retrieve/pii/S0098135422002228.

Breiman, L., 2001. Random Forests. Mach. Learn. 45 (1), 5–32. http://dx.doi.org/10.1023/A:1010933404324.

Camacho, J., Pérez-Villegas, A., Rodríguez-Gómez, R.A., Jiménez-Mañas, E., 2015. Multivariate exploratory data analysis (MEDA) toolbox for matlab. Chemometr. Intell. Lab. Syst. 143, 49–57. http://dx.doi.org/10.1016/j.chemolab.2015.02.016, URL: https://linkinghub.elsevier.com/retrieve/pii/S0169743915000465.

Chiang, L.H., Russell, E.L., Braatz, R.D., 2001. Fault Detection and Diagnosis in Industrial Systems, Softcover reprint of the original 1st ed. 2001 ed. Springer, London ; New York.

Downs, J., Vogel, E., 1993. A plant-wide industrial process control problem. Comput. Chem. Eng. 17 (3), 245–255. http://dx.doi.org/10.1016/0098-1354(93)80018-I.

Fontoura, T.B., de Sá, M.C.C., de Menezes, D.Q.F., Oechsler, B.F., Melo, A., de O. Campos, L.F., Anzai, T.K., Diehl, F.C., Thompson, P.H., Pinto, J.C., 2022. Modeling of spiral wound membranes for gas separations. Part III: A nonisothermal 2D permeation model. Chem. Eng. Res. Des. 177, 376–393. http://dx.doi.org/10.1016/j.cherd.2021.10.036, URL: https://www.sciencedirect.com/science/article/pii/S0263876221004469.

González-Martínez, J., Camacho, J., Ferrer, A., 2018. MVBatch: A matlab toolbox for batch process modeling and monitoring. Chemometr. Intell. Lab. Syst. 183, 122–133. http://dx.doi.org/10.1016/j.chemolab.2018.11.001, URL: https://linkinghub.elsevier.com/retrieve/pii/S0169743918304003.

Jiang, Y., Yin, S., 2019. Recent advances in key-performance-indicator oriented prognosis and diagnosis with a MATLAB toolbox: DB-KIT. IEEE Trans. Ind. Inform. 15 (5), 2849–2858. http://dx.doi.org/10.1109/TII.2018.2875067, URL: https://ieeexplore.ieee.org/document/8486739/.

Jiang, Y., Yin, S., Yang, Y., 2016. Comparison of KPI related fault detection algorithms using a newly developed MATLAB toolbox: DB-KIT. In: IECON 2016 - 42nd Annual Conference of the IEEE Industrial Electronics Society. IEEE, Florence, Italy, pp. 7149–7154. http://dx.doi.org/10.1109/IECON.2016.7792957, URL: http://ieeexplore.ieee.org/document/7792957/.

Lemos, T., Campos, L.F., Melo, A., Clavijo, N., Soares, R., Câmara, M., Feital, T., Anzai, T., Pinto, J.C., 2021. Echo state network based soft sensor for monitoring and fault detection of industrial processes. Comput. Chem. Eng. 155, 107512. http://dx.doi.org/10.1016/j.compchemeng.2021.107512, URL: https://www.sciencedirect.com/science/article/pii/S0098135421002908.

Lian, V., Varoy, E., Giacaman, N., 2022. Learning object-oriented programming concepts through visual analogies. IEEE Trans. Learn. Technol. 15 (1), 78–92. http://dx.doi.org/10.1109/TLT.2022.3154805, URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9722963.

Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS '17, Curran Associates Inc., Red Hook, NY, USA, pp. 4768–4777.

Marins, M.A., Ribeiro, F.M.L., Netto, S.L., da Silva, E.A.B., 2018. Improved similarity-based modeling for the classification of rotating-machine failures. J. Franklin Inst. 355 (4), 1913–1930. http://dx.doi.org/10.1016/j.jfranklin.2017.07.038, URL: https://www.sciencedirect.com/science/article/pii/S0016003217303678.

McKinney, W., 2010. Data structures for statistical computing in python. In: van der Walt, S., Millman, J. (Eds.), Proceedings of the 9th Python in Science Conference. pp. 51–56.

Melo, A., Câmara, M.M., Clavijo, N., Pinto, J.C., 2022. Open benchmarks for assessment of process monitoring and fault diagnosis techniques: A review and critical analysis. Comput. Chem. Eng. 165, 107964. http://dx.doi.org/10.1016/j.compchemeng.2022.107964, URL: https://www.sciencedirect.com/science/article/pii/S0098135422003003.

Melo, A., Câmara, M.M., Pinto, J.C., 2024. Data-driven process monitoring and fault diagnosis: A comprehensive survey. Processes 12 (2), 251. http://dx.doi.org/10.3390/pr12020251, URL: https://www.mdpi.com/2227-9717/12/2/251.

Melo, A., Clavijo, N., Soares, R., Lemos, T., Diehl, F., Anzai, T., Thompson, P., Pinto, J.C., 2023. BibMon: Biblioteca de Monitoramento de Processos. Instituto Nacional de Propriedade Intelectual (INPI) Register BR512023001653-5.

Montgomery, D.C., 2024. The 100th anniversary of the control chart. J. Qual. Technol. 56 (1), 2–4. http://dx.doi.org/10.1080/00224065.2023.2282926, URL: https://www.tandfonline.com/doi/full/10.1080/00224065.2023.2282926.

Pazel, D.P., 2022. Software modeling and object-oriented design. In: Pazel, D.P. (Ed.), Music Representation and Transformation in Software: Structure and Algorithms in Python. Springer International Publishing, Cham, pp. 3–10. http://dx.doi.org/10.1007/978-3-030-97472-5_1.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., 2011. Scikit-learn: Machine learning in python. J. Mach. Learn. Res. 12, 2825–2830, URL: http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf.

Perera, Y.S., Ratnaweera, D.A.A.C., Dasanayaka, C.H., Abeykoon, C., 2023. The role of artificial intelligence-driven soft sensors in advanced sustainable process industries: A critical review. Eng. Appl. Artif. Intell. 121, 105988. http://dx.doi.org/10.1016/j.engappai.2023.105988, URL: https://www.sciencedirect.com/science/article/pii/S0952197623001720.

Ribeiro, M.T., Singh, S., Guestrin, C., 2016. "Why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '16, Association for Computing Machinery, New York, NY, USA, pp. 1135–1144. http://dx.doi.org/10.1145/2939672.2939778, URL: https://dl.acm.org/doi/10.1145/2939672.2939778.

Rumbaugh, J., Jacobson, I., Booch, G., 2005. The unified modeling language reference manual. Addison-Wesley Object Technology Series, Addison-Wesley, number v. 1. URL: https://books.google.com.br/books?id=0a5QAAAAMAAJ.

Schaeffer, J., Braatz, R.D., 2022. Latent Variable Method Demonstrator — software for understanding multivariate data analytics algorithms. Comput. Chem. Eng. 167, 108014. http://dx.doi.org/10.1016/j.compchemeng.2022.108014, URL: https://www.sciencedirect.com/science/article/pii/S0098135422003489.

Sebesta, R.W., 2018. Concepts of Programming Languages, twelfth e.d Pearson, NY, NY.

Shi, L., Liang, X., 2011. An object-oriented approach for structure design of property calculation programs using equations of State. Ind. Eng. Chem. Res. 50 (10), 6404–6412. http://dx.doi.org/10.1021/ie101547t.

Spina, D.E., de Oliveira Campos, L.F., de Arruda, W.F., Melo, A., Alves, M.d.S., Rabello, G.L., Anzai, T.K., Pinto, J.C., 2024. Comparison of autoencoder architectures for fault detection in industrial processes. http://dx.doi.org/10.2139/ssrn.4758647, URL: https://papers.ssrn.com/abstract=4758647.

Thibault, E., Chioua, M., McKay, M., Korbel, M., Patience, G.S., Stuart, P.R., 2023. Experimental methods in chemical engineering: Data processing and data usage in decision-making. Can. J. Chem. Eng. cjce.25014. http://dx.doi.org/10.1002/cjce.25014, URL: https://onlinelibrary.wiley.com/doi/10.1002/cjce.25014.

Vargas, R.E.V., Munaro, C.J., Ciarelli, P.M., Medeiros, A.G., Amaral, B.G.d., Barrionuevo, D.C., Araújo, J.C.D.d., Ribeiro, J.L., Magalhães, L.P., 2019. A realistic and public dataset with rare undesirable real events in oil wells. J. Pet. Sci. Eng. 181, http://dx.doi.org/10.1016/j.petrol.2019.106223, URL: https://linkinghub.elsevier.com/retrieve/pii/S0920410519306357.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M., 2020. HuggingFace's transformers: State-of-the-art natural language processing. http://dx.doi.org/10.48550/arXiv.1910.03771, URL: http://arxiv.org/abs/1910.03771, arXiv:1910.03771 [cs].

Yi, G., Herdsman, C., Morris, J., 2019. A MATLAB toolbox for data preprocessing and multivariate statistical process control. Chemometr. Intell. Lab. Syst. 194, 103863. http://dx.doi.org/10.1016/j.chemolab.2019.103863, URL: https://linkinghub.elsevier.com/retrieve/pii/S0169743919304551.

Afrânio Melo is a Data Scientist at Petrobras, where he works on R&D in petroleum production engineering, including topics such as production monitoring, flow assurance, multiphase flow simulation, and artificial lift methods. He received his doctoral and master's degrees in Chemical Engineering from UFRJ in 2023 and 2016, respectively. He worked as a Researcher at Engepol and a Lecturer at UFRJ and CEDERJ. In addition to technical work, he practices literary writing, having won the Hernâni Cidade Literary Prize in Portugal in 2021. For additional information, visit: https://afraniomelo.github.io/en.